计算几何：二维竞赛板子

默认 C++17。目标是赛场可直接复制、接口短、常数小；整数几何用 `Point<ll>` 做精确判定，涉及交点、距离、圆、旋转时用 `Pd`。

0. 先看这里：约定与选型

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ld = long double;

const ld PI = acosl(-1.0L);
const ld EPS = 1e-12L;
const ld INF = numeric_limits<ld>::infinity();

#define Lt Line<T>
#define Pt Point<T>
#define Pd Point<ld>
#define Ld Line<ld>
#define cc(x) cout << fixed << setprecision(x);
```

推荐选择：

场景	类型 / 算法	复杂度
共线、方向、线段相交	`Point<ll>` + 叉积	`O(1)`
交点、投影、距离、圆	`Pd` / `Ld`	`O(1)`
任意多边形点包含	射线法	单次 `O(n)`
静态凸包	Andrew	`O(n log n)`
点在凸包内	扇形二分	单次 `O(log n)`
凸包直径	旋转卡壳	`O(n)`
最近点对	分治	`O(n log n)`
最小圆覆盖	随机增量	期望 `O(n)`
两凸包相加	闵可夫斯基和	`O(n+m)`
多个半平面公共区域	双端队列	`O(n log n)`
矩形面积并	扫描线	`O(n log n)`
多圆面积并	圆弧区间合并	`O(n^2 log n)`

场景	类型 / 算法	复杂度
固定半径圆最多覆盖点	角度扫描	`O(n^2 log n)`
最小面积包围矩形	旋转卡壳	`O(n)`
凸多边形交集	逐边裁剪	`O(nm)`

数值规则：

- `sgn(x)` 才用于几何符号判断；不要直接写 `x==0` 判断浮点数。
- `Point::operator<` 和 `operator==` 是精确比较，保证 `sort/set` 的严格弱序。浮点点判等用 `same(a,b)`。
- `Point<ll>` 的乘法仍可能溢出。若坐标绝对值接近 `1e9` 且有多次乘法，至少把关键式转成 `__int128`；文末提供了安全叉积。
- `EPS` 与题目尺度有关。坐标约 `1e9`、误差要求较松时可改为 `1e-10`；高精度输出通常用 `1e-12`。
- 多边形默认顶点按逆时针排列、不重复首点；线 `Line{a,b}` 表示从 `a` 指向 `b`。

---

1. 基础层：点、向量、直线

这一节是后续所有模块的依赖，建议整体复制。

- `Point<T>(x,y)`：建立点或向量。
- `Line<T>(a,b)`：建立过 `a,b` 的有向直线；作线段函数参数时表示闭线段 `[a,b]`。
- `dot(a,b)`、`cross(a,b)`：点积、叉积。
- `cross(o,a,b)`：即 `(a-o) x (b-o)`，正数表示 `o->a->b` 左转。
- `norm2(a)`：向量长度平方；`dis2(a,b)`：两点距离平方。
- `rot90(a)`：逆时针旋转 90 度；`rot(a,r)`：逆时针旋转 `r` 弧度。

基础运算均为 `O(1)`。

```
template<class T> int sgn(T x) { // x: 待判符号的数
    if constexpr (is_floating_point_v<T>) return (x > EPS) - (x < -EPS);
    else return (x > 0) - (x < 0);
}
template<class A, class B> bool eq(A x, B y) { // 浮点近似相等
    return fabsl((ld)x - (ld)y) <= EPS;
}
template<class T> T sq(T x) { return x * x; }
ld clamp1(ld x) { return max(-1.0L, min(1.0L, x)); }

template<class T> struct Point {
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}
    template<class U> explicit operator Point<U>() const { return {(U)x, (U)y}; }
```

```
    Pt& operator+=(const Pt& p) { x += p.x, y += p.y; return *this; }
    Pt& operator-=(const Pt& p) { x -= p.x, y -= p.y; return *this; }
    Pt& operator*=(T k) { x *= k, y *= k; return *this; }
    Pt& operator/=(T k) { x /= k, y /= k; return *this; }
    Pt operator-() const { return {-x, -y}; }
    friend Pt operator+(Pt a, const Pt& b) { return a += b; }
    friend Pt operator-(Pt a, const Pt& b) { return a -= b; }
    friend Pt operator*(Pt a, T k) { return a *= k; }
    friend Pt operator*(T k, Pt a) { return a *= k; }
    friend Pt operator/(Pt a, T k) { return a /= k; }
    friend bool operator<(const Pt& a, const Pt& b) {
        return a.x != b.x ? a.x < b.x : a.y < b.y;
    }
    friend bool operator==(const Pt& a, const Pt& b) { return a.x == b.x && a.y ==
b.y; }
    friend bool operator!=(const Pt& a, const Pt& b) { return !(a == b); }
    friend istream& operator>>(istream& in, Pt& p) { return in >> p.x >> p.y; }
    friend ostream& operator<<(ostream& out, const Pt& p) { return out << p.x << ' '
<< p.y; }
};

template<class T> struct Line {
    Pt a, b;
    Line(Pt a = {}, Pt b = {}) : a(a), b(b) {}
};

template<class T> T dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
template<class T> T cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
template<class T> T cross(Pt o, Pt a, Pt b) { return cross(a - o, b - o); }
template<class T> T norm2(Pt a) { return dot(a, a); }
template<class T> T dis2(Pt a, Pt b) { return norm2(a - b); }
template<class T> ld len(Pt a) { return sqrtl((ld)norm2(a)); }
template<class T> ld dis(Pt a, Pt b) { return len(a - b); }
template<class T> Pt rot90(Pt a) { return {-a.y, a.x}; }

bool same(Pd a, Pd b) { return eq(a.x, b.x) && eq(a.y, b.y); }
Pd toPd(Point<ll> p) { return {(ld)p.x, (ld)p.y}; }
Pd unit(Pd a) { return a / len(a); } // a: 非零向量
Pd rot(Pd a, ld r) { // a: 向量; r: 逆时针弧度
    ld c = cosl(r), s = sinl(r);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}
Pd rot(Pd p, Pd o, ld r) { return o + rot(p - o, r); } // p 绕 o 逆时针转 r
ld angle(Pd a, Pd b) { return atan2l(fabsl(cross(a, b)), dot(a, b)); } // [0,PI]
```

1.1 极角排序（无 `atan2`）

按从正 x 轴开始逆时针排序，射线相同则短向量在前。整数点全程精确。若要绕点 `o` 排序，先存 `p-o`，排完再加回 `o`。

```cpp
template<class T> int half(Pt p) { // 0: 上半平面及正 x 轴
    return p.y < 0 || (p.y == 0 && p.x < 0);
}
template<class T> bool polarCmp(Pt a, Pt b) { // a,b: 相对极点的非零向量
    int x = half(a), y = half(b);
    if (x != y) return x < y;
    T c = cross(a, b);
    return c != 0 ? c > 0 : norm2(a) < norm2(b);
}
template<class T> bool angleCmp(Pt a, Pt b) { // 仅比较方向，同向向量视为等价
    int x=half(a),y=half(b);
    return x!=y?x<y:cross(a,b)>0;
}
```

1.2 大坐标安全叉积

当输入为 `ll` 且乘积可能超过 `9e18` 时，用 `cross128` 做符号判定。它返回 `__int128_t`，无需输出。

```cpp
using i128 = __int128_t;
i128 cross128(Point<ll> a, Point<ll> b) { // 两个 ll 向量
    return (i128)a.x * b.y - (i128)a.y * b.x;
}
i128 cross128(Point<ll> o, Point<ll> a, Point<ll> b) { // o->a 与 o->b
    return cross128(a - o, b - o);
}
```

---

2. 直线与线段

2.1 位置、投影、距离

- `onLine(p,l)`：点是否在无限直线上。
- `onSeg(p,s)`：点是否在闭线段上，端点算在线段上。
- `lineCross(a,b,p)`：求两无限直线交点，平行返回 `false`；输入必须是 `Ld`。
- `proj(p,l)` / `refl(p,l)`：点到直线的投影 / 关于直线的对称点。
- `disLine` / `disSeg`：点到无限直线 / 闭线段距离。

所有函数 `O(1)`。直线两端必须不同。

```cpp
template<class T> bool onLine(Pt p, Lt l) { // p: 点; l: 无限直线
    return sgn(cross(l.a, l.b, p)) == 0;
}
template<class T> bool onSeg(Pt p, Lt s) { // p: 点; s: 闭线段
    return sgn(cross(s.a, s.b, p)) == 0 && sgn(dot(p - s.a, p - s.b)) <= 0;
}
template<class T> bool parallel(Lt a, Lt b) { return sgn(cross(a.b-a.a, b.b-b.a)) ==
0; }
```

```
template<class T> bool perpendicular(Lt a, Lt b) { return sgn(dot(a.b-a.a, b.b-b.a))
== 0; }

bool lineCross(Ld a, Ld b, Pd& p) { // a,b: 无限直线; p: 输出交点
    Pd u = a.b - a.a, v = b.b - b.a;
    ld d = cross(u, v);
    if (sgn(d) == 0) return false;
    p = a.a + u * (cross(b.a - a.a, v) / d);
    return true;
}
Pd proj(Pd p, Ld l) { // p: 点; l: 非退化无限直线
    Pd v = l.b - l.a;
    return l.a + v * (dot(p - l.a, v) / norm2(v));
}
Pd refl(Pd p, Ld l) { return proj(p, l) * 2 - p; }
ld disLine(Pd p, Ld l) { return fabsl(cross(l.b-l.a, p-l.a)) / len(l.b-l.a); }
ld disSeg(Pd p, Ld s) { // p 到闭线段 s
    Pd v = s.b - s.a;
    if (sgn(norm2(v)) == 0) return dis(p, s.a);
    if (sgn(dot(p-s.a, v)) < 0) return dis(p, s.a);
    if (sgn(dot(p-s.b, v)) > 0) return dis(p, s.b);
    return fabsl(cross(v, p-s.a)) / len(v);
}
Pd nearSeg(Pd p, Ld s) { // 返回 p 在线段 s 上的最近点
    Pd v = s.b - s.a;
    if (sgn(norm2(v)) == 0) return s.a;
    ld t = dot(p-s.a, v) / norm2(v);
    t = max(0.0L, min(1.0L, t));
    return s.a + v * t;
}
Ld perpendicularBisector(Pd a, Pd b) { // a,b: 不同点; 返回中垂线
    Pd m=(a+b)/2;
    return {m,m+rot90(b-a)};
}
Ld angleBisector(Pd o, Pd a, Pd b) { // 角 a-o-b 的内角平分线，夹角不能为 PI
    Pd v=unit(a-o)+unit(b-o);
    return {o,o+v};
}
```

2.2 线段相交与距离

`segCross(a,b)` 判断两个闭线段是否有公共点，包括端点接触和共线重叠，适合只问“是否相交”。整数坐标时是精确判定。

`segCrossPts(a,b)` 进一步返回交集：空数组表示不交；一个点表示单点相交；两个点表示重叠线段的两个端点。此函数输出浮点点。

```
template<class T> bool segCross(Lt a, Lt b) { // a,b: 闭线段
    T c1 = cross(a.a, a.b, b.a), c2 = cross(a.a, a.b, b.b);
    T c3 = cross(b.a, b.b, a.a), c4 = cross(b.a, b.b, a.b);
    if (sgn(c1) == 0 && onSeg(b.a, a)) return true;
```

```
    if (sgn(c2) == 0 && onSeg(b.b, a)) return true;
    if (sgn(c3) == 0 && onSeg(a.a, b)) return true;
    if (sgn(c4) == 0 && onSeg(a.b, b)) return true;
    return sgn(c1) * sgn(c2) < 0 && sgn(c3) * sgn(c4) < 0;
}

vector<Pd> segCrossPts(Ld a, Ld b) { // a,b: 闭线段; 返回 0/1/2 个点
    Pd p;
    if (lineCross(a, b, p)) return onSeg(p, a) && onSeg(p, b) ? vector<Pd>{p} :
vector<Pd>{};
    if (sgn(cross(a.b-a.a, b.a-a.a)) != 0) return {};
    vector<Pd> v;
    for (Pd q : {a.a, a.b, b.a, b.b}) if (onSeg(q, a) && onSeg(q, b))
v.push_back(q);
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end(), same), v.end());
    if (v.size() > 2) v = {v.front(), v.back()};
    return v;
}

ld disSegSeg(Ld a, Ld b) { // 两闭线段最短距离
    if (segCross(a, b)) return 0;
    return min({disSeg(a.a,b), disSeg(a.b,b), disSeg(b.a,a), disSeg(b.b,a)});
}
```

2.3 直线一般式

`ABC(a,b)` 把两点式转为 `Ax+By+C=0`，返回 `{A,B,C}`。`fromABC(A,B,C)` 反向构造任意两个点；要求 `A,B` 不同时为零。

```
template<class T> array<T,3> ABC(Lt l) { // l: 两点确定的直线
    T A = l.a.y-l.b.y, B = l.b.x-l.a.x;
    return {A, B, -(A*l.a.x+B*l.a.y)};
}
Ld fromABC(ld A, ld B, ld C) { // Ax+By+C=0
    if (sgn(B)) return {{0,-C/B}, {1,-(A+C)/B}};
    return {{-C/A,0}, {-C/A,1}};
}
```

2.4 射线

射线 `Line{a,b}` 表示从端点 `a` 出发、方向为 `b-a`，要求 `a!=b`。

- `onRay(p,r)`：点是否在射线上，端点算在内；整数坐标可精确判断。
- `raySegCross(r,s)`：射线与闭线段是否有公共点，包括共线重叠。
- `rayCross(a,b)`：两条射线是否有公共点，包括共线同向、反向接触。

后两个函数使用 `Ld`，复杂度均为 `O(1)`。

```
template<class T> bool onRay(Pt p, Lt r) { // p: 点; r.a: 端点; r.a->r.b: 方向
```

```
    return sgn(cross(r.b-r.a,p-r.a))==0 && sgn(dot(p-r.a,r.b-r.a))>=0;
}
bool raySegCross(Ld r, Ld s) { // r: 射线; s: 闭线段
    Pd p;
    if (lineCross(r,s,p)) return onRay(p,r)&&onSeg(p,s);
    if (sgn(cross(r.b-r.a,s.a-r.a))!=0) return false;
    return onRay(s.a,r)||onRay(s.b,r)||onSeg(r.a,s);
}
bool rayCross(Ld a, Ld b) { // a,b: 两条射线
    Pd p;
    if (lineCross(a,b,p)) return onRay(p,a)&&onRay(p,b);
    if (sgn(cross(a.b-a.a,b.a-a.a))!=0) return false;
    return onRay(a.a,b)||onRay(b.a,a);
}
```

---

3. 圆与三角形

统一圆结构 `Circle{o,r}`。除"点与圆的位置关系"外，返回交点的函数都使用 `ld`。

```
struct Circle {
    Pd o;
    ld r;
    Circle(Pd o = {}, ld r = 0) : o(o), r(r) {}
};
int rel(Pd p, Circle c) { return sgn(dis2(p,c.o)-c.r*c.r); } // -1 内,0 上,1 外
Pd onCircle(Circle c, ld a) { return c.o + Pd(cosl(a),sinl(a))*c.r; } // a: 弧度
ld sector(ld r, ld a) { return r*r*a/2; } // a: 圆心角弧度，可带符号
```

3.1 直线 / 线段与圆

`lineCircle(l,c)` 返回无限直线与圆的 0、1、2 个交点。`segCircle(s,c)` 只保留闭线段上的交点。结果无顺序保证。

```
vector<Pd> lineCircle(Ld l, Circle c) { // l: 非退化无限直线; c: 圆
    Pd p = proj(c.o, l), v = unit(l.b-l.a);
    ld h2 = c.r*c.r-dis2(p,c.o);
    if (sgn(h2) < 0) return {};
    if (sgn(h2) == 0) return {p};
    Pd d = v*sqrtl(max(0.0L,h2));
    return {p-d,p+d};
}
vector<Pd> segCircle(Ld s, Circle c) { // s: 闭线段
    vector<Pd> r;
    for (Pd p : lineCircle(s,c)) if (onSeg(p,s)) r.push_back(p);
    return r;
}
```

3.2 两圆位置与交点

`circleRel(a,b)` 返回：`-2` 内含无交点，`-1` 内切，`0` 两点相交，`1` 外切，`2` 相离，`3` 重合。`circleCross` 返回 0、1、2 个交点；重合圆也返回空数组，因为交点无限多。

```
int circleRel(Circle a, Circle b) { // a,b: 两圆
    ld d = dis(a.o,b.o), sm = a.r+b.r, df = fabsl(a.r-b.r);
    if (sgn(d)==0 && sgn(a.r-b.r)==0) return 3;
    if (sgn(d-sm)>0) return 2;
    if (sgn(d-sm)==0) return 1;
    if (sgn(d-df)>0) return 0;
    if (sgn(d-df)==0) return -1;
    return -2;
}
vector<Pd> circleCross(Circle a, Circle b) { // 返回有限交点
    Pd v = b.o-a.o;
    ld d = len(v);
    if (sgn(d)==0 || sgn(d-a.r-b.r)>0 || sgn(d-fabsl(a.r-b.r))<0) return {};
    ld x = (a.r*a.r-b.r*b.r+d*d)/(2*d);
    ld h2 = a.r*a.r-x*x;
    Pd p = a.o+v*(x/d);
    if (sgn(h2)==0) return {p};
    Pd q = rot90(v)*(sqrtl(max(0.0L,h2))/d);
    return {p-q,p+q};
}
```

3.3 点到圆的切点

`tangentPts(p,c)` 返回从点 `p` 向圆 `c` 作切线的切点：圆内为空，圆上只有自身，圆外有两个。若要切线，连接 `p` 与返回点即可。

```
vector<Pd> tangentPts(Pd p, Circle c) { // p: 点; c: 圆
    Pd v = p-c.o;
    ld d2 = norm2(v), r2 = c.r*c.r;
    if (sgn(d2-r2)<0) return {};
    if (sgn(d2-r2)==0) return {p};
    Pd q = c.o+v*(r2/d2), w = rot90(v)*(c.r*sqrtl(d2-r2)/d2);
    return {q-w,q+w};
}
```

3.4 两圆公切线

`commonTangents(a,b)` 返回公切线，每条线的 `a` 点在第一个圆上、`b` 点在第二个圆上。通常相离圆有 4 条，外切 3 条，相交 2 条，内切 1 条，内含 0 条；重合圆有无数条，此处返回空。

```
vector<Ld> commonTangents(Circle a, Circle b) { // a,b: 两圆
    vector<Ld> z;
    Pd d = b.o-a.o;
    ld d2 = norm2(d);
```

```
    if (sgn(d2)==0) return z;
    for (int s : {-1,1}) { // s=1 外公切线, s=-1 内公切线
        ld r = a.r-s*b.r, h2 = d2-r*r;
        if (sgn(h2)<0) continue;
        h2 = max(0.0L,h2);
        for (int t : {-1,1}) {
            Pd v = (d*r+rot90(d)*(sqrtl(h2)*t))/d2;
            z.push_back({a.o+v*a.r, b.o+v*(s*b.r)});
            if (sgn(h2)==0) break;
        }
    }
    return z;
}
```

3.5 两圆交面积

`circleOverlap(a,b)` 返回两个圆盘的交面积。内含、相离、相切均已处理，复杂度 `O(1)`。

```
ld circleOverlap(Circle a, Circle b) { // 两圆盘交面积
    ld d = dis(a.o,b.o);
    if (sgn(d-a.r-b.r)>=0) return 0;
    if (sgn(d-fabsl(a.r-b.r))<=0) {
        ld r = min(a.r,b.r);
        return PI*r*r;
    }
    ld x = acosl(clamp1((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r)));
    ld y = acosl(clamp1((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r)));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sinl(x);
}
```

3.6 三角形面积与常用中心

输入三点不能共线。`circumcircle` 返回外接圆；`incenter` 返回内心；`orthocenter` 返回垂心。三角形有向二倍面积直接用 `cross(a,b,c)`。

```
ld triArea(Pd a, Pd b, Pd c) { return fabsl(cross(a,b,c))/2; }
Circle circumcircle(Pd a, Pd b, Pd c) { // a,b,c: 不共线
    Pd u=b-a, v=c-a;
    ld d=2*cross(u,v);
    Pd o=a+Pd(v.y*norm2(u)-u.y*norm2(v), u.x*norm2(v)-v.x*norm2(u))/d;
    return {o,dis(o,a)};
}
Circle diameterCircle(Pd a, Pd b) { return {(a+b)/2,dis(a,b)/2}; } // ab 为直径
Circle circle3(Pd a, Pd b, Pd c) { // 覆盖三点的最小圆, 允许共线
    if (sgn(cross(a,b,c))!=0) return circumcircle(a,b,c);
    if (dis2(a,b)>=dis2(a,c)&&dis2(a,b)>=dis2(b,c)) return diameterCircle(a,b);
    if (dis2(a,c)>=dis2(b,c)) return diameterCircle(a,c);
    return diameterCircle(b,c);
}
Pd incenter(Pd a, Pd b, Pd c) { // 三角形内心
```

```
    ld x=dis(b,c), y=dis(a,c), z=dis(a,b);
    return (a*x+b*y+c*z)/(x+y+z);
}
Circle incircle(Pd a, Pd b, Pd c) { // 三角形内切圆
    Pd o=incenter(a,b,c);
    return {o,disLine(o,{a,b})};
}
Pd orthocenter(Pd a, Pd b, Pd c) { // 三角形垂心
    Circle q=circumcircle(a,b,c);
    return a+b+c-q.o*2;
}
```

点在三角形内与重心坐标

`inTri(p,a,b,c)` 返回 `0` 外、`1` 边界、`2` 内部，三角形方向不限；要求三点不共线。`barycentric` 返回点 `p` 关于 `a,b,c` 的重心坐标 `{u,v,w}`，满足 `p=u*a+v*b+w*c` 且 `u+v+w=1`。三个坐标均非负等价于点在闭三角形内。

```
template<class T> int inTri(Pt p, Pt a, Pt b, Pt c) { // p: 点; abc: 非退化三角形
    int x=sgn(cross(a,b,p)),y=sgn(cross(b,c,p)),z=sgn(cross(c,a,p));
    if ((x<0||y<0||z<0)&&(x>0||y>0||z>0)) return 0;
    return x==0||y==0||z==0?1:2;
}
array<ld,3> barycentric(Pd p, Pd a, Pd b, Pd c) { // p 关于三角形 abc
    ld s=cross(a,b,c);
    return {cross(p,b,c)/s,cross(p,c,a)/s,cross(p,a,b)/s};
}
Pd triCentroid(Pd a, Pd b, Pd c) { return (a+b+c)/3; } // 三角形重心
```

3.7 最小圆覆盖

求覆盖所有点的最小圆。先随机打乱，再做随机增量；期望 `O(n)`，最坏 `O(n^3)`。返回圆心和半径。空集返回半径 `0` 的原点圆。

```
Circle minCoverCircle(vector<Pd> p) { // p: 任意点集，会被打乱
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(p.begin(),p.end(),rng);
    Circle c{{0,0},-1};
    auto out=[&](Pd x){ return c.r<0 || sgn(dis2(x,c.o)-c.r*c.r)>0; };
    for (int i=0;i<(int)p.size();i++) if (out(p[i])) {
        c={p[i],0};
        for (int j=0;j<i;j++) if (out(p[j])) {
            c=diameterCircle(p[i],p[j]);
            for (int k=0;k<j;k++) if (out(p[k])) c=circle3(p[i],p[j],p[k]);
        }
    }
    if (c.r<0) c={{0,0},0};
    return c;
}
```

3.8 定半径圆经过两点

`circleCenters(a,b,r)` 返回所有半径为 `r` 且经过 `a,b` 的圆心：距离大于 `2r` 时无解，等于 `2r` 时一个解，小于时两个解。`a==b` 时有无穷多解，本函数返回空数组，调用方必须单独处理。

```
vector<Pd> circleCenters(Pd a, Pd b, ld r) { // a,b: 两点; r: 非负半径
    ld d=dis(a,b);
    if (sgn(d)==0||sgn(d-2*r)>0) return {};
    Pd m=(a+b)/2;
    ld h2=r*r-d*d/4;
    if (sgn(h2)==0) return {m};
    Pd v=unit(rot90(b-a))*sqrtl(max(0.0L,h2));
    return {m-v,m+v};
}
```

3.9 多圆面积并

`circleUnionArea(c)` 求若干圆盘的并面积。对每个圆，把被其他圆覆盖的圆周角区间合并，再用 Green 公式累加未覆盖圆弧贡献。支持包含、相切、重复圆；复杂度 `O(n^2 log n)`。

这里求的是“至少被一个圆覆盖”的面积。恰好覆盖 `k` 次的面积需要圆弧事件再增加覆盖层数统计，不是本函数的返回值。

```
ld arcArea(Circle c, ld a, ld b) { // 圆 c 上从角 a 到 b 的逆时针弧贡献
    return (c.r*c.r*(b-a)+c.r*(c.o.x*(sinl(b)-sinl(a))-c.o.y*(cosl(b)-cosl(a))))/2;
}
ld circleUnionArea(const vector<Circle>& c) { // c: 圆盘集合
    ld ans=0; int n=c.size();
    for (int i=0;i<n;i++) {
        bool dead=false;
        vector<pair<ld,ld>> v;
        for (int j=0;j<n;j++) if (i!=j) {
            ld d=dis(c[i].o,c[j].o);
            if (sgn(d)==0&&sgn(c[i].r-c[j].r)==0) {
                if (i>j) dead=true;
                continue;
            }
            if (sgn(d+c[i].r-c[j].r)<=0) { dead=true; break; }
            if (sgn(d-c[i].r-c[j].r)>=0||sgn(d-fabsl(c[i].r-c[j].r))<=0) continue;
            ld a=atan2l(c[j].o.y-c[i].o.y,c[j].o.x-c[i].o.x);
            if (a<0) a+=2*PI;
            ld da=acosl(clamp1((c[i].r*c[i].r+d*d-c[j].r*c[j].r)/(2*c[i].r*d)));
            ld l=a-da,r=a+da;
            while (l<0) l+=2*PI,r+=2*PI;
            while (l>=2*PI) l-=2*PI,r-=2*PI;
            if (r<=2*PI) v.push_back({l,r});
            else v.push_back({l,2*PI}),v.push_back({0,r-2*PI});
        }
        if (dead) continue;
        sort(v.begin(),v.end());
```

```
        ld pre=0;
        for (int k=0;k<(int)v.size();) {
            ld l=v[k].first,r=v[k].second; int j=k+1;
            while (j<(int)v.size()&&v[j].first<=r+EPS) r=max(r,v[j++].second);
            if (pre<l-EPS) ans+=arcArea(c[i],pre,l);
            pre=max(pre,r),k=j;
        }
        if (pre<2*PI-EPS) ans+=arcArea(c[i],pre,2*PI);
    }
    return fabsl(ans);
}
```

3.10 固定半径圆最多覆盖点

`maxCoverCircle(p,r)` 返回半径为 `r` 的某个闭圆盘最多能覆盖多少输入点。固定一个点在圆周上，其他点对圆心方向形成角区间，然后扫描端点；复杂度 `O(n^2 log n)`，空间 `O(n)`。

重复点会分别计数。`r=0` 时答案是完全相同点的最大重数。

```
int maxCoverCircle(const vector<Pd>& p, ld r) { // p: 点集; r: 固定半径
    int n=p.size(),ans=0;
    if (sgn(r)==0) {
        for (int i=0;i<n;i++) { int z=0; for (int j=0;j<n;j++) z+=same(p[i],p[j]);
ans=max(ans,z); }
        return ans;
    }
    for (int i=0;i<n;i++) {
        int base=1;
        vector<pair<ld,int>> e;
        for (int j=0;j<n;j++) if (i!=j) {
            ld d=dis(p[i],p[j]);
            if (sgn(d)==0) { base++; continue; }
            if (sgn(d-2*r)>0) continue;
            ld a=atan2l(p[j].y-p[i].y,p[j].x-p[i].x);
            if (a<0) a+=2*PI;
            ld da=acosl(clamp1(d/(2*r))),l=a-da,rr=a+da;
            while (l<0) l+=2*PI,rr+=2*PI;
            while (l>=2*PI) l-=2*PI,rr-=2*PI;
            if (rr<=2*PI) e.push_back({l,1}),e.push_back({rr,-1});
            else {
                e.push_back({l,1}),e.push_back({2*PI,-1});
                e.push_back({0,1}),e.push_back({rr-2*PI,-1});
            }
        }
        sort(e.begin(),e.end(),[](auto a,auto b){ return a.first!=b.first?
a.first<b.first:a.second>b.second; });
        int z=base; ans=max(ans,z);
        for (auto [a,t]:e) z+=t,ans=max(ans,z);
    }
    return ans;
}
```

4. 任意简单多边形

顶点默认按边界顺序给出，顺时针或逆时针均可；无需在末尾重复首点。简单多边形指边除相邻端点外不自交。

4.1 面积、周长、重心

- `area2(p)`：有向二倍面积；逆时针为正。
- `area(p)`：无向面积。
- `perimeter(p)`：周长。
- `centroid(p)`：均匀薄片的面积重心，要求面积非零。

均为 `O(n)`。

```
template<class T> T area2(const vector<Pt>& p) { // p: 边界有序顶点
    T s=0; int n=p.size();
    for (int i=0;i<n;i++) s+=cross(p[i],p[(i+1)%n]);
    return s;
}
template<class T> ld area(const vector<Pt>& p) { return fabsl((ld)area2(p))/2; }
template<class T> ld perimeter(const vector<Pt>& p) {
    ld s=0; int n=p.size();
    for (int i=0;i<n;i++) s+=dis(p[i],p[(i+1)%n]);
    return s;
}
Pd centroid(const vector<Pd>& p) { // p: 面积非零的简单多边形
    Pd g{}; ld s=0; int n=p.size();
    for (int i=0;i<n;i++) {
        ld c=cross(p[i],p[(i+1)%n]);
        g+=(p[i]+p[(i+1)%n])*c, s+=c;
    }
    return g/(3*s);
}
```

方向归一化与凸性判断

`makeCCW(p)` 把边界方向改成逆时针。`isConvex(p,strict)` 判断边界有序多边形是否凸：`strict=true` 不允许连续三点共线，`false` 允许边上冗余点。自交输入不属于该接口前提；少于三个点或全部共线返回 `false`。

```
template<class T> void makeCCW(vector<Pt>& p) { // p: 边界有序多边形
    if (sgn(area2(p))<0) reverse(p.begin(),p.end());
}
template<class T> bool isConvex(const vector<Pt>& p, bool strict=true) { // p: 简单多边形
    int n=p.size(),d=0;
    if (n<3) return false;
    for (int i=0;i<n;i++) {
```

```
        int s=sgn(cross(p[i],p[(i+1)%n],p[(i+2)%n]));
        if (!s) { if (strict) return false; else continue; }
        if (d&&d!=s) return false;
        d=s;
    }
    return d!=0;
}
```

4.2 点在任意多边形内

`inPoly(p,a)` 返回 `0` 外部、`1` 边界、`2` 内部。射线法不要求凸，多边形方向不限，单次 `O(n)`。

```
template<class T> int inPoly(Pt p, const vector<Pt>& a) { // p: 查询点; a: 简单多边形
    bool in=0; int n=a.size();
    for (int i=0;i<n;i++) {
        Pt u=a[i], v=a[(i+1)%n];
        if (onSeg(p,Lt{u,v})) return 1;
        bool f=(u.y>p.y)!=(v.y>p.y);
        if (f && sgn(cross(v-u,p-u))==(v.y>u.y?1:-1)) in=!in;
    }
    return in?2:0;
}
```

4.3 网格多边形与 Pick 定理

只用于整数格点简单多边形。

- `gridBorder(p)`：边界上的格点数。每条边贡献 `gcd(|dx|,|dy|)`，不会重复计顶点。
- `gridInside(p)`：内部格点数，由 `2S=2I+B-2` 得到。

复杂度 `O(n log C)`。

```
ll gridBorder(const vector<Point<ll>>& p) { // p: 整数格点多边形
    ll b=0; int n=p.size();
    for (int i=0;i<n;i++) {
        auto d=p[(i+1)%n]-p[i];
        b+=gcd(llabs(d.x),llabs(d.y));
    }
    return b;
}
ll gridInside(const vector<Point<ll>>& p) { // 要求 area2 不溢出 ll
    ll a=llabs(area2(p)), b=gridBorder(p);
    return (a-b+2)/2;
}
```

4.4 直线裁剪多边形

保留有向直线 `l.a -> l.b` 左侧（含边界）的部分。输入可为任意凸/凹简单多边形，但单次裁剪后仍按边界顺序输出。常用于 Sutherland-Hodgman 多边形裁剪；连续按若干直线裁剪即可求凸区域交。

```
vector<Pd> cutPoly(const vector<Pd>& p, Ld l) { // p: 多边形; l: 保留左侧
    vector<Pd> q; int n=p.size();
    for (int i=0;i<n;i++) {
        Pd a=p[i], b=p[(i+1)%n];
        int x=sgn(cross(l.b-l.a,a-l.a)), y=sgn(cross(l.b-l.a,b-l.a));
        if (x>=0) q.push_back(a);
        if ((x<0&&y>0)||(x>0&&y<0)) {
            Pd z; lineCross({a,b},l,z); q.push_back(z);
        }
    }
    return q;
}
```

4.5 圆与多边形交面积

`circlePolyArea(p,c)` 求圆盘与简单多边形内部的交面积，输入方向不限，复杂度 `O(n)`。核心是把每条边平移到圆心后，计算三角形 `Oab` 与圆的有向交面积。

```
ld edgeCircleArea(Pd a, Pd b, ld r) { // 有向边 a->b，圆心在原点
    Pd d=b-a; ld A=norm2(d), B=2*dot(a,d), C=norm2(a)-r*r;
    vector<ld> t={0,1};
    ld D=B*B-4*A*C;
    if (sgn(D)>0) {
        D=sqrtl(D);
        for (ld x : {(-B-D)/(2*A),(-B+D)/(2*A)}) if (x>EPS&&x<1-EPS) t.push_back(x);
    }
    sort(t.begin(),t.end());
    ld s=0;
    for (int i=0;i+1<(int)t.size();i++) {
        Pd u=a+d*t[i], v=a+d*t[i+1], m=(u+v)/2;
        if (sgn(norm2(m)-r*r)<=0) s+=cross(u,v)/2;
        else s+=r*r*atan2l(cross(u,v),dot(u,v))/2;
    }
    return s;
}
ld circlePolyArea(const vector<Pd>& p, Circle c) { // 圆盘与简单多边形交面积
    ld s=0; int n=p.size();
    for (int i=0;i<n;i++) s+=edgeCircleArea(p[i]-c.o,p[(i+1)%n]-c.o,c.r);
    return fabsl(s);
}
```

5. 凸包与凸多边形

5.1 Andrew 静态凸包

`convexHull(p,strict)` 返回逆时针凸包，不重复首点。

- `strict=true`：删除边上的中间点，只保留凸包顶点。通常旋转卡壳、凸包二分用这个。
- `strict=false`：保留边界共线点。若所有点共线，会按正序和逆序重复中间点，因此全共线需求建议自行去重后直接排序。

函数会排序并去重，复杂度 `O(n log n)`。

```
template<class T> vector<Pt> convexHull(vector<Pt> p, bool strict=true) { // p: 点集
    sort(p.begin(),p.end());
    p.erase(unique(p.begin(),p.end()),p.end());
    int n=p.size(), k=0;
    if (n<=1) return p;
    vector<Pt> h(2*n);
    auto bad=[&](Pt a,Pt b,Pt c){ int s=sgn(cross(a,b,c)); return strict?s<=0:s<0;
};
    for (auto x:p) { while (k>=2&&bad(h[k-2],h[k-1],x)) k--; h[k++]=x; }
    for (int i=n-2,t=k+1;i>=0;i--) {
        while (k>=t&&bad(h[k-2],h[k-1],p[i])) k--;
        h[k++]=p[i];
    }
    h.resize(k-1);
    return h;
}
```

5.2 点在凸包内：`O(log n)`

`inConvex(p,h)` 返回 `0` 外、`1` 边界、`2` 内。要求 `h` 是严格凸包、逆时针、不重复首点；长度 1、2 也能处理。

```
template<class T> int inConvex(Pt p, const vector<Pt>& h) { // p: 点; h: 严格逆时针凸包
    int n=h.size();
    if (!n) return 0;
    if (n==1) return p==h[0]?1:0;
    if (n==2) return onSeg(p,Lt{h[0],h[1]})?1:0;
    int a=sgn(cross(h[0],h[1],p)), b=sgn(cross(h[0],h[n-1],p));
    if (a<0||b>0) return 0;
    if (a==0) return onSeg(p,Lt{h[0],h[1]})?1:0;
    if (b==0) return onSeg(p,Lt{h[0],h[n-1]})?1:0;
    int l=1,r=n-1;
    while (r-l>1) { int m=(l+r)/2; (sgn(cross(h[0],h[m],p))>=0?l:r)=m; }
    int s=sgn(cross(h[l],h[(l+1)%n],p));
    return s<0?0:(s==0?1:2);
}
```

支持点 / 最大点积查询

`ConvexQuery<T>` 对严格逆时针凸包预处理。`extreme(v)` 返回使 `dot(p,v)` 最大的顶点在内部数组 `p` 中的下标，`maxDot(v)` 直接返回最大点积；方向 `v` 必须非零。预处理 `O(n)`，单次查询 `O(log n)`。

构造时会循环平移顶点起点，因此返回下标对应 `q.p[id]`，不是原输入数组下标。最大值落在整条边上时返回其中一个端点。

```
template<class T> struct ConvexQuery {
    vector<Pt> p,e;
    ConvexQuery(vector<Pt> h={}) : p(move(h)) { // h: 严格逆时针凸包
        int n=p.size();
        if (n<=1) return;
        e.resize(n);
        for (int i=0;i<n;i++) e[i]=p[(i+1)%n]-p[i];
        int k=min_element(e.begin(),e.end(),[](Pt a,Pt b){ return angleCmp<T>(a,b);
})-e.begin();
        rotate(p.begin(),p.begin()+k,p.end());
        for (int i=0;i<n;i++) e[i]=p[(i+1)%n]-p[i];
    }
    int extreme(Pt v) const { // v: 非零方向; 最大化 dot(p,v)
        int n=p.size();
        if (n<=1) return 0;
        Pt d=rot90(v);
        auto it=lower_bound(e.begin(),e.end(),d,[](Pt a,Pt b){ return angleCmp<T>
(a,b); });
        return it==e.end()?0:int(it-e.begin());
    }
    T maxDot(Pt v) const { return dot(p[extreme(v)],v); } // v: 非零方向
};
```

5.3 旋转卡壳：凸包直径

`hullDiameter2(h)` 返回凸包任意两点最大距离的平方，不开根号。要求严格逆时针凸包。复杂度 `O(n)`；整数坐标可精确计算。

```
template<class T> T hullDiameter2(const vector<Pt>& h) { // h: 严格逆时针凸包
    int n=h.size();
    if (n<=1) return 0;
    if (n==2) return dis2(h[0],h[1]);
    T ans=0; int j=1;
    for (int i=0;i<n;i++) {
        int ni=(i+1)%n;
        while (cross(h[ni]-h[i],h[(j+1)%n]-h[i])>cross(h[ni]-h[i],h[j]-h[i])) j=
(j+1)%n;
        ans=max({ans,dis2(h[i],h[j]),dis2(h[ni],h[j])});
    }
    return ans;
}
```

5.4 旋转卡壳：最小宽度

`hullWidth(h)` 返回能夹住凸包的两条平行支撑线的最小距离。要求严格逆时针凸包，复杂度 `O(n)`。

```
ld hullWidth(const vector<Pd>& h) { // h: 严格逆时针凸包
    int n=h.size();
    if (n<=2) return 0;
    ld ans=INF; int j=1;
    for (int i=0;i<n;i++) {
        int ni=(i+1)%n;
        while (cross(h[ni]-h[i],h[(j+1)%n]-h[i])>cross(h[ni]-h[i],h[j]-h[i])+EPS) j=
(j+1)%n;
        ans=min(ans,fabsl(cross(h[ni]-h[i],h[j]-h[i]))/len(h[ni]-h[i]));
    }
    return ans;
}
```

5.5 最小面积包围矩形

`minAreaRect(h)` 求包含整个凸包的最小面积矩形，同时返回周长和四个顶点。要求 `h` 是严格逆时针凸包。四组支撑线随边旋转，三个对踵点只会单向移动，复杂度 `O(n)`。

返回顶点按矩形边界顺序排列。点集退化为一个点或一条线段时面积为 `0`。

```
struct MinRect {
    ld area=0, peri=0;
    array<Pd,4> p{};
};
MinRect minAreaRect(const vector<Pd>& h) { // h: 严格逆时针凸包
    int n=h.size();
    if (!n) return {};
    if (n==1) return {0,0,{h[0],h[0],h[0],h[0]}};
    MinRect ans; ans.area=INF;
    int j=0,k=0,l=0;
    Pd e0=h[1]-h[0];
    for (int q=1;q<n;q++) { // 为首条边初始化三条支撑线，之后指针单调移动
        if (cross(e0,h[q]-h[0])>cross(e0,h[j]-h[0])) j=q;
        if (dot(e0,h[q])>dot(e0,h[k])) k=q;
        if (dot(e0,h[q])<dot(e0,h[l])) l=q;
    }
    for (int i=0;i<n;i++) {
        int ni=(i+1)%n; Pd e=h[ni]-h[i];
        while (cross(e,h[(j+1)%n]-h[i])>cross(e,h[j]-h[i])+EPS) j=(j+1)%n;
        while (dot(e,h[(k+1)%n])>dot(e,h[k])+EPS) k=(k+1)%n;
        while (dot(e,h[(l+1)%n])<dot(e,h[l])-EPS) l=(l+1)%n;
        Pd u=unit(e),v=rot90(u);
        ld x1=dot(u,h[l]),x2=dot(u,h[k]);
        ld y1=dot(v,h[i]),y2=dot(v,h[j]);
        ld ar=(x2-x1)*(y2-y1),pe=2*((x2-x1)+(y2-y1));
        if (ar<ans.area-EPS) ans={ar,pe,{u*x1+v*y1,u*x2+v*y1,u*x2+v*y2,u*x1+v*y2}};
    }
```

```
    return ans;
}
```

5.6 凸包内最大面积三角形

`maxTriArea2(h)` 返回从凸包顶点中选三个点能得到的最大二倍面积。要求严格逆时针凸包，复杂度 `O(n^2)`，整数坐标可精确计算；普通面积再除以 `2`。

```
template<class T> T maxTriArea2(const vector<Pt>& h) { // h: 严格逆时针凸包
    int n=h.size(); T ans=0;
    for (int i=0;i<n;i++) {
        int k=i+2;
        for (int j=i+1;j+1<n;j++) {
            k=max(k,j+1);
            while (k+1<n&&cross(h[i],h[j],h[k+1])>=cross(h[i],h[j],h[k])) k++;
            ans=max(ans,cross(h[i],h[j],h[k]));
        }
    }
    return ans;
}
```

5.7 闵可夫斯基和

求集合 `{a+b | a in A, b in B}` 的凸包。输入 `a,b` 必须是严格逆时针凸包；函数内部会旋转到字典序最小点开头。返回严格凸包。除最后的清理外合并过程 `O(n+m)`，整体 `O((n+m) log(n+m))` 是因为调用了通用凸包去冗余；输入严格时实际点数很小且常数低。

```
template<class T> vector<Pt> minkowski(vector<Pt> a, vector<Pt> b) { // a,b: 严格逆时针凸包
    if (a.empty()||b.empty()) return {};
    if (a.size()==1) { for (auto& p:b) p+=a[0]; return b; }
    if (b.size()==1) { for (auto& p:a) p+=b[0]; return a; }
    auto prep=[](vector<Pt>& p) {
        rotate(p.begin(),min_element(p.begin(),p.end()),p.end());
        p.push_back(p[0]);
    };
    prep(a),prep(b);
    int n=a.size()-1,m=b.size()-1,i=0,j=0;
    vector<Pt> c={a[0]+b[0]};
    while (i<n||j<m) {
        Pt x=i<n?a[i+1]-a[i]:Pt{}, y=j<m?b[j+1]-b[j]:Pt{};
        int s=(i==n)?-1:(j==m?1:sgn(cross(x,y)));
        if (s>=0) c.push_back(c.back()+x),i++;
        if (s<=0) c.back()+=y,j++;
    }
    c.pop_back();
    return convexHull(c,true);
}
```

5.8 两凸多边形是否相交：闵可夫斯基差

`convexDisjoint(a,b)` 判断两个闭凸多边形是否完全不相交。构造 `A+(-B)` 后判断原点；边界接触也算相交，因此仅当返回 `true` 时才无公共点。

```
template<class T> bool convexDisjoint(vector<Pt> a, vector<Pt> b) { // a,b: 严格逆时针凸包
    for (auto& p:b) p=-p;
    auto c=minkowski(a,b);
    return inConvex(Pt{},c)==0;
}
```

5.9 两凸多边形交集

`convexIntersection(a,b)` 返回两个闭凸多边形的交集顶点。输入可以顺时针或逆时针，函数会把裁剪多边形转为逆时针；复杂度 `O(nm)`，适合通常规模。若 `n,m` 都极大，应改用线性凸多边形求交专题算法。

返回少于三个点时交集面积为零；当前裁剪接口主要面向有正面积的交集，只有点或线段的退化交集可能返回空或重复点。

```
vector<Pd> convexIntersection(vector<Pd> a, vector<Pd> b) { // a,b: 凸多边形
    if (a.empty()||b.empty()) return {};
    if (sgn(area2(b))<0) reverse(b.begin(),b.end());
    for (int i=0;i<(int)b.size()&&!a.empty();i++) a=cutPoly(a,
{b[i],b[(i+1)%b.size()]});
    return a;
}
```

5.10 两凸多边形最短距离

`convexDistance(a,b)` 返回两个闭凸多边形的最短欧氏距离。输入要求是严格逆时针凸包；内部构造闵可夫斯基差，若原点不在差中，再求原点到其边界的距离。复杂度由当前 `minkowski` 的末尾去冗余决定，为 `O((n+m)log(n+m))`。

```
ld convexDistance(vector<Pd> a, vector<Pd> b) { // a,b: 严格逆时针凸包
    if (a.empty()||b.empty()) return INF;
    for (auto& p:b) p=-p;
    auto c=minkowski(a,b);
    if (inConvex(Pd{},c)) return 0;
    if (c.size()==1) return len(c[0]);
    ld ans=INF;
    for (int i=0;i<(int)c.size();i++)
ans=min(ans,disSeg(Pd{},Ld{c[i],c[(i+1)%c.size()]}));
    return ans;
}
```

6. 最近点对

求平面点集中最近两点距离。分治版 `closestPair(p)` 返回实际距离；若不足两个点返回 `INF`。函数会按 x 排序并在递归中按 y 维护，复杂度 `O(n log n)`，空间 `O(n)`。

若输入是整数且必须输出距离平方，可把内部 `ld` 改为 `ll`、所有距离换成 `dis2`，条带判断也比较平方；当前版本更短，适合一般浮点/整数输入。

```
ld closestPair(vector<Pd> p) { // p: 点集
    int n=p.size();
    if (n<2) return INF;
    sort(p.begin(),p.end());
    vector<Pd> t(n);
    function<ld(int,int)> go=[&](int l,int r)->ld {
        if (r-l<=3) {
            ld d=INF;
            for (int i=l;i<r;i++) for (int j=i+1;j<r;j++) d=min(d,dis(p[i],p[j]));
            sort(p.begin()+l,p.begin()+r,[](Pd a,Pd b){ return a.y<b.y; });
            return d;
        }
        int m=(l+r)/2; ld x=p[m].x;
        ld d=min(go(l,m),go(m,r));
        merge(p.begin()+l,p.begin()+m,p.begin()+m,p.begin()+r,t.begin(),[](Pd a,Pd
b){ return a.y<b.y; });
        copy(t.begin(),t.begin()+r-l,p.begin()+l);
        int k=0;
        for (int i=l;i<r;i++) if (fabsl(p[i].x-x)<d) {
            for (int j=k-1;j>=0&&p[i].y-t[j].y<d;j--) d=min(d,dis(p[i],t[j]));
            t[k++]=p[i];
        }
        return d;
    };
    return go(0,n);
}
```

6.1 整数精确平方版

`closestPair2(p)` 返回 `Point<ll>` 点集的最近距离平方，不开根号、不引入浮点误差；不足两个点返回 `LLONG_MAX`。递归同时维护 y 序，复杂度 `O(n log n)`。

要求任意两点距离平方能装入 `ll`。条带筛选使用 `i128` 避免坐标差平方在比较时先溢出。

```
ll closestPair2(vector<Point<ll>> p) { // p: ll 点集; 返回最近距离平方
    int n=p.size();
    if (n<2) return LLONG_MAX;
    sort(p.begin(),p.end());
    vector<Point<ll>> t(n);
    function<ll(int,int)> go=[&](int l,int r)->ll {
        if (r-l<=3) {
            ll d=LLONG_MAX;
            for (int i=l;i<r;i++) for (int j=i+1;j<r;j++) d=min(d,dis2(p[i],p[j]));
```

```
            sort(p.begin()+l,p.begin()+r,[](auto a,auto b){ return a.y<b.y; });
            return d;
        }
        int m=(l+r)/2; ll x=p[m].x;
        ll d=min(go(l,m),go(m,r));
        merge(p.begin()+l,p.begin()+m,p.begin()+m,p.begin()+r,t.begin(),[](auto
a,auto b){ return a.y<b.y; });
        copy(t.begin(),t.begin()+r-l,p.begin()+l);
        int k=0;
        for (int i=l;i<r;i++) {
            ll dx=p[i].x-x;
            if ((i128)dx*dx>=d) continue;
            for (int j=k-1;j>=0;j--) {
                ll dy=p[i].y-t[j].y;
                if ((i128)dy*dy>=d) break;
                d=min(d,dis2(p[i],t[j]));
            }
            t[k++]=p[i];
        }
        return d;
    };
    return go(0,n);
}
```

7. 半平面交

每条有向直线 `a->b` 表示保留其左侧（含边界）。`halfPlane(ls)` 返回所有半平面的有界公共区域，顶点逆时针排列；空集或无界区域均返回空，因此处理无界交时先加入一个足够大的外包矩形。

角度排序后用双端队列维护，复杂度 `O(n log n)`。平行同向直线只保留更靠左、约束更紧的一条。

```
int dirHalf(Pd v) { return v.y<0 || (sgn(v.y)==0&&v.x<0); }
bool hpOut(Pd p, Ld l) { return sgn(cross(l.b-l.a,p-l.a))<0; }

vector<Pd> halfPlane(vector<Ld> l) { // l: 每条线保留左半平面
    sort(l.begin(),l.end(),[](Ld a,Ld b) {
        Pd x=a.b-a.a,y=b.b-b.a;
        int u=dirHalf(x),v=dirHalf(y);
        if (u!=v) return u<v;
        ld c=cross(x,y);
        return sgn(c)!=0?c>0:cross(x,b.a-a.a)>0;
    });
    vector<Ld> v;
    for (auto x:l) {
        if (!v.empty()&&sgn(cross(v.back().b-v.back().a,x.b-x.a))==0) {
            if (hpOut(v.back().a,x)) v.back()=x;
        } else v.push_back(x);
    }
    deque<Ld> q;
    deque<Pd> p;
```

```
    for (auto x:v) {
        while (!p.empty()&&hpOut(p.back(),x)) p.pop_back(),q.pop_back();
        while (!p.empty()&&hpOut(p.front(),x)) p.pop_front(),q.pop_front();
        if (!q.empty()) {
            Pd z;
            if (!lineCross(q.back(),x,z)) return {};
            p.push_back(z);
        }
        q.push_back(x);
    }
    while (!p.empty()&&hpOut(p.back(),q.front())) p.pop_back(),q.pop_back();
    while (!p.empty()&&hpOut(p.front(),q.back())) p.pop_front(),q.pop_front();
    if (q.size()<3) return {};
    Pd z;
    if (!lineCross(q.back(),q.front(),z)) return {};
    p.push_back(z);
    return {p.begin(),p.end()};
}
```

7.1 简单多边形的核

多边形的核是能看见多边形内所有点的位置集合。对逆时针简单多边形，每条边的内侧都是左半平面，因此核就是所有边对应半平面的交。

`polygonKernel(p)` 返回核多边形，空数组表示没有核；输入方向不限。复杂度 `O(n log n)`。它依赖本节 `halfPlane`，不需要另加外包框，因为简单多边形本身有界。

```
vector<Pd> polygonKernel(vector<Pd> p) { // p: 简单多边形边界
    if (p.size()<3) return {};
    if (sgn(area2(p))<0) reverse(p.begin(),p.end());
    vector<Ld> l;
    for (int i=0;i<(int)p.size();i++) l.push_back({p[i],p[(i+1)%p.size()]});
    return halfPlane(l);
}
```

加入外包框时要保证方向使内部在左侧。例如边长范围 `[-M,M]`：

```
void addBox(vector<Ld>& l, ld M) { // l: 半平面数组; M: 足够大的正数
    Pd a{-M,-M},b{M,-M},c{M,M},d{-M,M};
    l.push_back({a,b}),l.push_back({b,c}),l.push_back({c,d}),l.push_back({d,a});
}
```

---

8. 矩形面积并：扫描线

`rectUnion(rs)` 求多个轴平行矩形的面积并。每个矩形是 `{x1,y1,x2,y2}`，允许端点顺序颠倒；退化矩形自动忽略。扫描 x，线段树维护 y 方向被覆盖总长度，复杂度 `O(n log n)`，空间 `O(n)`。

整数输入返回 `ll`。若坐标差和面积可能溢出 `ll`，把 `ans` 和乘法改成 `i128`。

```cpp
struct Rect { ll x1,y1,x2,y2; };

ll rectUnion(vector<Rect> r) { // r: 轴平行矩形
    struct E { ll x,y1,y2; int v; };
    vector<E> e; vector<ll> y;
    for (auto a:r) {
        if (a.x1>a.x2) swap(a.x1,a.x2);
        if (a.y1>a.y2) swap(a.y1,a.y2);
        if (a.x1==a.x2||a.y1==a.y2) continue;
        e.push_back({a.x1,a.y1,a.y2,1}); e.push_back({a.x2,a.y1,a.y2,-1});
        y.push_back(a.y1),y.push_back(a.y2);
    }
    if (e.empty()) return 0;
    sort(y.begin(),y.end()); y.erase(unique(y.begin(),y.end()),y.end());
    sort(e.begin(),e.end(),[](E a,E b){ return a.x<b.x; });
    int m=y.size()-1;
    vector<int> c(4*m+4); vector<ll> s(4*m+4);
    function<void(int,int,int,int,int,int)> add=[&](int o,int l,int r,int ql,int
qr,int v) {
        if (ql<=l&&r<=qr) c[o]+=v;
        else { int m=(l+r)/2; if (ql<=m) add(o*2,l,m,ql,qr,v); if (qr>m)
add(o*2+1,m+1,r,ql,qr,v); }
        if (c[o]) s[o]=y[r+1]-y[l];
        else s[o]=l==r?0:s[o*2]+s[o*2+1];
    };
    ll ans=0,px=e[0].x;
    for (int i=0;i<(int)e.size();) {
        ll x=e[i].x; ans+=s[1]*(x-px);
        while (i<(int)e.size()&&e[i].x==x) {
            int l=lower_bound(y.begin(),y.end(),e[i].y1)-y.begin();
            int r=lower_bound(y.begin(),y.end(),e[i].y2)-y.begin()-1;
            if (l<=r) add(1,0,m-1,l,r,e[i].v);
            i++;
        }
        px=x;
    }
    return ans;
}
```

9. 坐标变换、距离模型与反演

9.1 曼哈顿与切比雪夫距离

令 `u=x+y, v=x-y`，对两点坐标差有：

- 曼哈顿距离 `|dx|+|dy| = max(|du|,|dv|)`；
- 切比雪夫距离 `max(|dx|,|dy|) = (|du|+|dv|)/2`；
- 从 `(u,v)` 还原：`x=(u+v)/2, y=(u-v)/2`，注意整数点要求 `u,v` 奇偶性相同。

`manhattanDiameter(p)` 返回点集中最大曼哈顿距离，扫描 `x+y` 和 `x-y` 的极差即可，复杂度 `O(n)`。

```
template<class T> T manhattan(Pt a, Pt b) { return abs(a.x-b.x)+abs(a.y-b.y); }
template<class T> T chebyshev(Pt a, Pt b) { return max(abs(a.x-b.x),abs(a.y-b.y)); }
template<class T> T manhattanDiameter(const vector<Pt>& p) { // p: 非空点集
    T a=dot(p[0],Pt{1,1}),b=a,c=dot(p[0],Pt{1,-1}),d=c;
    for (auto q:p) {
        T x=q.x+q.y,y=q.x-q.y;
        a=min(a,x),b=max(b,x),c=min(c,y),d=max(d,y);
    }
    return max(b-a,d-c);
}
```

9.2 圆反演

以圆 `c` 为反演圆，点 `p` 的像 `p'` 与圆心共线且满足 `|op|*|op'|=r^2`。`invertPoint` 要求 `p` 不是反演中心。

- 不过反演中心的圆仍变成圆，使用 `invertCircle`。
- 不过反演中心的直线变成过反演中心的圆，使用 `invertLine`。
- 过反演中心的圆变成直线；过反演中心的直线保持自身，代码中不强行用错误类型表示这两种退化结果。
- 反演保持夹角大小但反转方向，常用于圆族相切、路径绕圆等问题。

```
Pd invertPoint(Pd p, Circle c) { // p: 非圆心点; c: 反演圆
    Pd v=p-c.o;
    return c.o+v*(c.r*c.r/norm2(v));
}
Circle invertCircle(Circle a, Circle c) { // a: 不过 c.o 的圆; c: 反演圆
    Pd v=a.o-c.o; ld d2=norm2(v),z=d2-a.r*a.r;
    return {c.o+v*(c.r*c.r/z),fabsl(c.r*c.r*a.r/z)};
}
Circle invertLine(Ld l, Circle c) { // l: 不过 c.o 的直线; c: 反演圆
    Pd h=invertPoint(proj(c.o,l),c);
    return diameterCircle(c.o,h);
}
```

9.3 圆幂与根轴

点 `p` 对圆的幂为 `|po|^2-r^2`：圆外为正、圆上为零、圆内为负。若从圆外点作切线，切线长平方等于圆幂。

两不共心圆的根轴是对两圆幂相等的点集，也就是公共弦所在直线；即使两圆不相交，根轴仍存在。`radicalAxis(a,b)` 要求圆心不同。

```
ld power(Pd p, Circle c) { return dis2(p,c.o)-c.r*c.r; } // p 对圆 c 的幂
Ld radicalAxis(Circle a, Circle b) { // a,b: 不共心圆
    ld A=2*(b.o.x-a.o.x),B=2*(b.o.y-a.o.y);
    ld z=norm2(b.o)-b.r*b.r-norm2(a.o)+a.r*a.r;
    return fromABC(A,B,-z);
}
```

10. 常用公式、定理与快速提醒

10.1 角度与三角形

- 弧度转角度：`deg = rad * 180 / PI`。
- 角度转弧度：`rad = deg * PI / 180`。
- 余弦定理：边 `a,b` 的夹角所对边为 `c`，则 `C=acos((a*a+b*b-c*c)/(2*a*b))`。传给 `acosl` 前用 `clamp1`。
- 正弦定理：`a/sin(A)=b/sin(B)=c/sin(C)=2R`。
- 海伦公式：`s=(a+b+c)/2`，面积 `sqrt(s(s-a)(s-b)(s-c))`。
- 三角形内切圆半径 `r=S/s`；外接圆半径 `R=abc/(4S)`。
- 三角形重心把每条中线按 `2:1` 分割；向量式 `G=(A+B+C)/3`。
- 外心 `O`、重心 `G`、垂心 `H` 共线，且 `OG:GH=1:2`，向量式 `H=A+B+C-2O`。
- 九点圆圆心是 `O,H` 的中点，半径为外接圆半径的一半。
- 角平分线定理：若 `AD` 平分角 `A`，则 `BD/DC=AB/AC`。
- Stewart 定理：三角形边 `a=m+n`，劈线长 `d`，则 `b^2m+c^2n=a(d^2+mn)`。

10.2 圆、弦与切线结论

- 弦长：圆心角为 `t` 时，弦长 `2r*sin(t/2)`，小弓形面积 `(r^2/2)*(t-sin(t))`。
- 相交弦：圆内两弦交于 `P`，则 `PA*PB=PC*PD`。
- 割线：圆外点 `P` 的两条割线满足 `PA*PB=PC*PD=power(P,c)`。
- 切割线：切线切点 `T` 与割线交点 `A,B` 满足 `PT^2=PA*PB`。
- 两圆根轴垂直于圆心连线；三个圆两两根轴若不平行，则共点（根心）。
- 圆周角等于所对圆心角的一半；同弧所对圆周角相等。

10.3 Pick 定理

顶点均为格点的简单多边形满足 `S=I+B/2-1`：`I` 是内部格点数，`B` 是边界格点数。注意它不适用于自交多边形或非格点顶点。

10.4 凸性与分离结论

- 有限点集在线性函数 `dot(p,v)` 下的最大值一定在凸包顶点上；多次查询可使用 `ConvexQuery`。
- 两个闭凸集不相交，当且仅当存在一条直线把它们严格分开；工程上可用闵可夫斯基差是否包含原点判断。
- 凸多边形直径一定由一对对踵点取得；最小宽度和最小面积包围矩形至少有一条边与凸包边平行。
- 简单多边形有核当且仅当各条有向边内侧半平面的交非空。
- Helly 定理（二维）：有限个凸集的交非空，当且仅当任意三个的交非空。

10.5 常见错误清单

- 叉积参数顺序错：本板子三点版固定是 `cross(o,a,b)=(a-o)x(b-o)`。
- 求直线交点前忘记排除平行；使用 `lineCross` 的布尔返回值。
- 凸包方向不满足要求：`convexHull(...,true)` 保证逆时针且严格，可直接供二分和卡壳使用。
- 把无限直线、射线、线段混用：`Line` 只是两个点，语义由调用函数决定。
- 浮点比较器中使用 `EPS`：这可能破坏 `sort` 的严格弱序。本板子排序比较器使用精确坐标或稳定的半平面规则。
- 整数除法截断：任何交点、投影、圆心都先转成 `Pd/Ld`。
- 面积符号误解：`area2` 有符号；需要普通面积调用 `area`。
- 输入点重复：静态凸包会去重；最近点对不去重，存在重复点时正确返回 `0`。
- 圆面积并未处理零半径圆的边界积分意义；零半径圆面积为零，建议输入前删掉。
- `convexIntersection` 的点/线退化交集不是其主要接口目标；只求是否接触时用线段相交或闵可夫斯基差。
- 反演中心没有有限像；调用 `invertPoint` 前必须排除圆心。

---

11. 最小复制组合与专题边界

按题目类型选模块，减少赛场代码量：

题型	复制章节
方向、线段相交	`0 + 1 + 2.2`
直线交点、距离	`0 + 1 + 2.1`
圆交点、切线	`0 + 1 + 2.1 + 3` 对应小节
多圆面积并 / 固定半径覆盖	`0 + 1 + 2.1 + 3.9/3.10`
任意多边形包含 / 面积	`0 + 1 + 2.1 + 4` 对应小节
凸包直径	`0 + 1 + 5.1 + 5.3`

题型	复制章节
最小面积包围矩形	`0 + 1 + 5.1 + 5.5`
多次点在凸包内	`0 + 1 + 2.1 + 5.1 + 5.2`
两凸多边形距离	`0 + 1 + 2.1 + 5.1 + 5.7 + 5.10`
半平面交	`0 + 1 + 2.1 + 7`
最近点对	`0 + 1 + 6`
最小圆覆盖	`0 + 1 + 3.6 + 3.7`
矩形面积并	`0 + 8`

这份板子中的代码块按文档顺序拼接可共同编译。实际提交时只复制依赖链上的章节，避免无关代码增加出错面。

以下内容没有混入本通用板子：Delaunay 三角剖分 / Voronoi 图、一般多边形布尔运算、直线与线段安排、动态凸包、三维几何。它们代码长、退化约定多，通常应作为独立专题板子；遇到对应题目时不要用这里的短函数拼凑替代。